

[프로그래밍 기초] AST 맛보기

[들어가기 앞서]

- 13일로 오해해서 제출이 하루 늦었습니다. 죄송합니다.
- 과제를 잘 수행하지 못했습니다.
 - 하지만 정말 최선을 다 했습니다.
 - 현재까지 진행된 과정까지 보고서로 정리해서 제출하겠습니다.
 - 부족한 부분들은 다른 과제들 수행 이후, 반드시 다시 시도하겠습니다.

[마주한 문제 & 해결한 방법]

- ast.json 직렬화 하기
 - 서칭을 해보니 Java, python, 웹 툴 등을 이용해서 직렬화, 혹은 역직렬화를 한다는 사실을 알았습니다.
 - 직렬화를 위한 다양한 방법들 중, 멘토님께서도 단순 노가다인, 직렬화는 GPT를 이용한다고 하셔서, GPT를 이용했습니다.
 - ast.json 내용이 너무 길어서 내용을 복사해서 넣을 수 없었습니다. 그래서 파일을 통채로 넣어 txt로 받아내어 해결하였습니다.
- ast.json 직렬화 내용 불러오기
 - txt로 받았기 때문에, 직렬화한 txt 파일 내용을 Ctrl + C, V 하여 코드에 넣고자 했습니다.
 - 그러나 직렬화한 내용이 너무 길어서 Ctrl + C, V 할 수 없었습니다.
 - 그래서 파일포인터를 이용해서 txt 파일을 열고 내용을 가져왔습니다.
- 직렬화한 내용이 출력되는 현상
 - 서칭을 해보니, Json_c.c 파일 내의 printf("reallocated : %s\n", str); 가문제였습니다.

```

#ifdef __JSONC_BODY__
json_value json_string_to_value(const char** json_message) {
    while (c = ((*json_message)++)) {
        switch (c) {
            case '\\':
                str[size] = '\\';
                goto JSON_STRBREAK;
            default:
                str[size] = ch;
        }
        size++;
        if((size+1) % JSON_STRBUFSIZE == 0){
            str = (char *)realloc(str, sizeof(char) * JSON_STRBUFSIZE);
            //printf("reallocated : %s\n", str);
            if(str == NULL){
                printf("string malloc error;\n");
                return jsonv;
            }
        }
    }
}

```

- 주석처리 외에는 도저히 방법을 찾을 수가 없어서 주석처리 하였습니다.
- 과제 해결을 위한 코드 작성
 - 코드를 진행하면 할 수록 부족한 부분들이 많이 보였습니다.
 - 계속 보완하고 수정하였으나, 파라미터를 출력하는 과정에서 막혔습니다.
 - 애초에 문제 해결을 위한 접근법이 잘못됐던 것 같습니다.
 - 자세한 내용은 코드 설명을 드리며 서술하겠습니다.

~~→ 이걸 작성하는 시점까지.. 파일 포인터를 이용해서 ast.json 파일을 불러와서 읽어내고, json 파일 생성까지만 성공해냈다. 그 이후는 감이 안 와서 헤메고 있는 중이다. 과연 잘 끝낼 수 있을까?~~

~~→ 풀어내는 방법은 알겠는데.. 파일을 못 불러와서 죽을맛이다.~~

[부분 코드 & 코드 설명]

- 파일 내용 불러오기

```

#include <stdio.h>
#include "json_c.c"
#include <string.h>
#include <stdlib.h>

int main(){

    FILE *fp;
    fp = fopen("please.txt","r");
    if(fp == NULL){
        printf("오류");
        return 0;
    }

    fseek(fp, 0, SEEK_END);
    long bufsiz = ftell(fp);
    fseek(fp, 0, SEEK_SET);

    char *string = (char *)malloc(bufsiz + 1);
    memset(string, 0, bufsiz + 1);
    fread(string, bufsiz, 1, fp);

    json_value json = json_create(string);
    json_value ext = json_get(json, "ext");

    int ext_obj_count = json_len(ext);
    int function_count = 0;

```

- please.txt 파일이 ast.json 내용을 직렬화한 파일입니다.
- 파일 포인터를 이용해서 please.txt를 읽기 전용으로 열어냈습니다.
- fseek 함수를 이용해서 글자수를 파악하고, 버퍼의 크기를 설정했습니다.
- please.txt 파일 내용을 string 변수에 넣고 읽어낸 뒤, json 값을 생성했습니다.
- 후술될 함수 개수를 세기 위해서 오브젝트 개수와, 함수의 개수를 체크하는 정수형 변수를 선언했습니다.
- 참고로 please.txt 파일의 형태는 아래와 같습니다.

[please.txt 파일의 일부]

```
{W"_nodetypeW": W"FileASTW", W"coordW": null, W"extW": [{W"_nodetypeW": W"DeclW", W"alignW": [], W"bitsizeW": null, W"coordW": W"target.c:1:6W", W"funspecW": [], W"initW": null, W"nameW": W"exitW", W"qualsW": [], W"storageW": [], W"typeW": {W"_nodetypeW": W"FuncDeclW", W"argsW": {W"_nodetypeW": W"ParamListW", W"coordW": W"target.c:0:1W", W"paramsW": [{W"_nodetypeW": W"TypenameW", W"alignW": null, W"coordW": W"target.c:0:1W", W"nameW": null, W"qualsW": [], W"typeW": {W"_nodetypeW": W"TypeDeclW", W"alignW": null, W"coordW": null, W"declnameW": null, W"qualsW": [], W"typeW": {W"_nodetypeW": W"IdentifierTypeW", W"coordW": W"target.c:1:11W", W"namesW": [W"intW"]}}]}, W"coordW": W"target.c:1:6W", W"typeW": {W"_nodetypeW": W"TypeDeclW", W"alignW": null, W"coordW": W"target.c:1:6W", W"declnameW": W"exitW", W"qualsW": [], W"typeW": {W"_nodetypeW": W"IdentifierTypeW", W"coordW": W"target.c:1:1W", W"namesW": [W"voidW"]}}]}, {W"_nodetypeW": W"DeclW", W"alignW": [], W"bitsizeW": null, W"coordW": W"target.c:2:5W", W"funspecW": [], W"initW": null, W"nameW": W"getcharW", W"qualsW": [], W"storageW": [], W"typeW": {W"_nodetypeW": W"FuncDeclW", W"argsW": {W"_nodetypeW": W"ParamListW", W"coordW": W"target.c:0:1W", W"paramsW": [{W"_nodetypeW": W"TypenameW", W"alignW": null, W"coordW": W"target.c:0:1W", W"nameW": null, W"qualsW": [], W"typeW": {W"_nodetypeW": W"TypeDeclW", W"alignW": null, W"coordW": null, W"declnameW": null, W"qualsW": [], W"typeW":
```

- 함수 개수 세기

```

for(int i = 0; i < ext_obj_count; i++){
    json_value obj = json_get(ext, i);

    char* _nodetype = json_get_string(obj, "_nodetype");
    char* Decl_name;
    char* FuncDef_name;

    if(strcmp(_nodetype, "Decl") == 0 ){
        json_value type = json_get(obj, "type");
        Decl_name = json_get_string(type, "_nodetype");

        if(strcmp(Decl_name, "FuncDecl") == 0 ){
            function_count++;
        }
    }
    else if(strcmp(_nodetype, "FuncDef") == 0 ){
        function_count++;
        json_value decl = json_get(obj, "decl");
        FuncDef_name = json_get_string(decl, "name");
    }
    if(strcmp(Decl_name, FuncDef_name) == 0 ){
        function_count--;
    } //아쉽게도(?) 겹치는 함수는 없음
}

printf("함수의 개수 %d\n", function_count);

```

→ 일단 무식하게 접근해보았습니다.

→ 모든 오브젝트에 접근하고, 조건문을 이용했습니다.

→ 기본 내장 함수를 사용했을 수도 있고, 선언한 함수와 정의한 함수가 같을 수도 있다고 생각했습니다.

→ 그래서 동일한 함수를 카운트하는 것을 막고자, 함수의 이름을 파싱해 비교해서 같은 이름은 제외하도록 코드를 짰습니다.

→ 아쉽게도 겹치는 함수는 없었습니다. (혹은 제가 잘못 코딩한 것일 수도 있습니다.)

- 함수 리턴 타입

```
int number_check = 0;
for(int i = 0; i < ext_obj_count; i++){
    json_value obj = json_get(ext, i);

    char* _nodetype = json_get_string(obj, "_nodetype");
    char* Decl_name;
    char* FuncDef_name;

    if(strcmp(_nodetype, "Decl") == 0 ){
        json_value type = json_get(obj, "type");
        Decl_name = json_get_string(type, "_nodetype");

        if(strcmp(Decl_name, "FuncDecl") == 0){
            number_check++;
            json_value type_type = json_get(type, "type");
            json_value type_type_type = json_get(type_type, "type");
            char *type_type_type_nodetype = json_get_string(type_type_type, "_nodetype");
            if(strcmp(type_type_type_nodetype, "IdentifierType") == 0){
                json_value type_type_type_return = json_get(type_type_type, "names");
                char* return_type_str = json_get_string(type_type_type_return, 0);

                printf("%d번째 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);
            } else{
                json_value type_type_type_type = json_get(type_type_type, "type");
                json_value return_type = json_get(type_type_type_type, "names");
                char *return_type_str = json_get_string(return_type, 0);
                printf("%d번째 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);
            }
        }
    }
    else if(strcmp(_nodetype, "FuncDef") == 0){
        number_check++;
        json_value decl = json_get(obj, "decl");
        json_value type = json_get(decl, "type");
        json_value type_type = json_get(type, "type");
        json_value type_type_type = json_get(type_type, "type");
        char *type_type_type_nodetype = json_get_string(type_type_type, "_nodetype");

        if(strcmp(type_type_type_nodetype, "IdentifierType") == 0){
            json_value type_type_type_return = json_get(type_type_type, "names");
            char* return_type_str = json_get_string(type_type_type_return, 0);

            printf("%d번째 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);
        } else{
            json_value type_type_type_type = json_get(type_type_type, "type");
            json_value return_type = json_get(type_type_type_type, "names");
            char *return_type_str = json_get_string(return_type, 0);
            printf("%d번째 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);
        }
    }
}
```

→ 리턴 타입 구하는 것도 정말 무식하게 했습니다.

→ json_value로 내부로 들어가서 파싱해왔습니다.

- 함수 파라미터 타입

→ 여기서 접근법이 잘못됨을 느꼈습니다.

→ 이렇게 되면 파라미터가 없는 경우와 같은 예외적인 함수에 대응할 수 없습니다.

→ 대응하고자 다시 위와 같은 방식으로 코드를 작성해도 또 다른 예외의 경우가 나와서 이런 방식으로는 접근할 수 없겠다고 생각했습니다.

[전체 코드]

```

#include <stdio.h>
#include "json.c.c"
#include <string.h>
#include <stdlib.h>

int main(){
    FILE *fp;
    fp = fopen("please.txt","r");
    if(fp == NULL){
        printf("오류");
        return 0;
    }

    fseek(fp, 0, SEEK_END);
    long bufsz = ftell(fp);
    fseek(fp, 0, SEEK_SET);

    char *string = (char *)malloc(bufsz + 1);
    memset(string, 0, bufsz + 1);
    fread(string, bufsz, 1, fp);

    json_value json = json_create(string);
    json_value ext = json_get(json, "ext");

    int ext_obj_count = json_len(ext);
    int function_count = 0;

    //
    for(int i = 0; i < ext_obj_count; i++){
        json_value obj = json_get(ext, i);

        char* _nodetype = json_get_string(obj, "_nodetype");
        char* Decl_name;
        char* FuncDef_name;

        if(strcmp(_nodetype, "Decl") == 0 ){
            json_value type = json_get(obj, "type");
            Decl_name = json_get_string(type, "_nodetype");

            if(strcmp(Decl_name, "FuncDecl") == 0){
                function_count++;
            }
        }
        else if(strcmp(_nodetype, "FuncDef") == 0){
            function_count++;
            json_value decl = json_get(obj, "decl");
            FuncDef_name = json_get_string(decl, "name");
        }
        if(strcmp(Decl_name, FuncDef_name) == 0){
            function_count--;
        }
    } //매함개도(?) 한지는 함수는 없음

    printf("함수의 개수 %d\n", function_count);

    //함수 리턴 타입
    int number_check = 0;
    for(int i = 0; i < ext_obj_count; i++){
        json_value obj = json_get(ext, i);

        char* _nodetype = json_get_string(obj, "_nodetype");
        char* Decl_name;
        char* FuncDef_name;

        if(strcmp(_nodetype, "Decl") == 0 ){
            json_value type = json_get(obj, "type");
            Decl_name = json_get_string(type, "_nodetype");

            if(strcmp(Decl_name, "FuncDecl") == 0){
                number_check++;
                json_value type_type = json_get(type, "type");
                json_value type_type_type = json_get(type_type, "type");
                char *type_type_type_nodetype = json_get_string(type_type_type, "_nodetype");
                if(strcmp(type_type_type_nodetype, "IdentifierType") == 0){
                    json_value type_type_type_return = json_get(type_type_type_type, "names");
                    char* return_type_str = json_get_string(type_type_type_return, 0);
                }

                printf("%d번재 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);

                //파라미터 타입
                json_value args = json_get(type, "args");
                json_value params = json_get(args, "params");
                int params_len = json_len(params);

                for(int i = 0; i < params_len; i++){
                    json_value params_element = json_get(params, i);
                    json_value params_type = json_get(params_element, "type");
                    json_value params_type_type = json_get(params_type, "type");
                    json_value params_type_type_names = json_get(params_type_type, "names");
                    char* param_name = json_get_string(params_type_type_names, 0);

                    printf("%d번재 함수의 파라미터 타입은 %s 입니다. \n", number_check, param_name);
                }
            }
            else if(
                json_value type_type_type_type = json_get(type_type_type_type, "type");
                json_value return_type = json_get(type_type_type_type_type, "names");
                char *return_type_str = json_get_string(return_type, 0);
                printf("%d번재 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);
            )
        }
        else if(strcmp(_nodetype, "FuncDef") == 0){
            number_check++;
            json_value decl = json_get(obj, "decl");
            json_value type = json_get(decl, "type");
            json_value type_type = json_get(type, "type");
            json_value type_type_type = json_get(type_type, "type");
            char *type_type_type_nodetype = json_get_string(type_type_type, "_nodetype");

            if(strcmp(type_type_type_nodetype, "IdentifierType") == 0){
                json_value type_type_type_return = json_get(type_type_type_type, "names");
                char* return_type_str = json_get_string(type_type_type_return, 0);

                printf("%d번재 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);
            }
            else if(
                json_value type_type_type_type = json_get(type_type_type_type, "type");
                json_value return_type = json_get(type_type_type_type_type, "names");
                char *return_type_str = json_get_string(return_type, 0);
                printf("%d번재 함수의 리턴 타입은 %s 입니다. \n", number_check, return_type_str);
            )
        }
    }

    return 0;
}

```


→ 함수의 개수, 리턴 타입, 파라미터 타입 일부분을 구할 수 있을 정도로 구현했습니다.

[실행 결과]

- 미완성인 파라미터 타입 구하는 코드를 제외한 결과

```
함수의 개수 42
1번째 함수의 리턴 타입은 void 입니다.
2번째 함수의 리턴 타입은 int 입니다.
3번째 함수의 리턴 타입은 void 입니다.
4번째 함수의 리턴 타입은 int 입니다.
5번째 함수의 리턴 타입은 int 입니다.
6번째 함수의 리턴 타입은 int 입니다.
7번째 함수의 리턴 타입은 char 입니다.
8번째 함수의 리턴 타입은 void 입니다.
9번째 함수의 리턴 타입은 void 입니다.
10번째 함수의 리턴 타입은 void 입니다.
11번째 함수의 리턴 타입은 int 입니다.
12번째 함수의 리턴 타입은 int 입니다.
13번째 함수의 리턴 타입은 void 입니다.
14번째 함수의 리턴 타입은 void 입니다.
15번째 함수의 리턴 타입은 int 입니다.
16번째 함수의 리턴 타입은 void 입니다.
17번째 함수의 리턴 타입은 void 입니다.
18번째 함수의 리턴 타입은 void 입니다.
19번째 함수의 리턴 타입은 int 입니다.
20번째 함수의 리턴 타입은 void 입니다.
21번째 함수의 리턴 타입은 int 입니다.
22번째 함수의 리턴 타입은 void 입니다.
23번째 함수의 리턴 타입은 void 입니다.
24번째 함수의 리턴 타입은 void 입니다.
25번째 함수의 리턴 타입은 void 입니다.
26번째 함수의 리턴 타입은 void 입니다.
27번째 함수의 리턴 타입은 int 입니다.
28번째 함수의 리턴 타입은 int 입니다.
29번째 함수의 리턴 타입은 void 입니다.
30번째 함수의 리턴 타입은 int 입니다.
31번째 함수의 리턴 타입은 int 입니다.
32번째 함수의 리턴 타입은 int 입니다.
33번째 함수의 리턴 타입은 int 입니다.
34번째 함수의 리턴 타입은 int 입니다.
35번째 함수의 리턴 타입은 int 입니다.
36번째 함수의 리턴 타입은 int 입니다.
37번째 함수의 리턴 타입은 int 입니다.
38번째 함수의 리턴 타입은 int 입니다.
39번째 함수의 리턴 타입은 void 입니다.
40번째 함수의 리턴 타입은 void 입니다.
41번째 함수의 리턴 타입은 void 입니다.
42번째 함수의 리턴 타입은 int 입니다.
* 터미널이 작업에서 다시 사용될니다. 닫으려면 아무 키나 누르세요.
```

- 코드를 제외하지 않은 결과

```
함수의 개수 42
1번째 함수의 리턴 타입은 void 입니다.
1번째 함수의 파라미터 타입은 int 입니다.
=====
2번째 함수의 리턴 타입은 int 입니다.
2번째 함수의 파라미터 타입은 void 입니다.
=====
3번째 함수의 리턴 타입은 void 입니다.
4번째 함수의 리턴 타입은 int 입니다.
4번째 함수의 파라미터 타입은 int 입니다.
=====
5번째 함수의 리턴 타입은 int 입니다.
json_get error : the first argument of json_get should be an array or an object (type : null)
json_get_last_index : the type of json_value is not a JSON_ARRAY nor a JSON_OBJECT6번째 함수의 리턴 타입은 int 입니다.
7번째 함수의 리턴 타입은 char 입니다.
8번째 함수의 리턴 타입은 void 입니다.
9번째 함수의 리턴 타입은 void 입니다.
10번째 함수의 리턴 타입은 void 입니다.
11번째 함수의 리턴 타입은 int 입니다.
12번째 함수의 리턴 타입은 int 입니다.
13번째 함수의 리턴 타입은 void 입니다.
14번째 함수의 리턴 타입은 void 입니다.
15번째 함수의 리턴 타입은 int 입니다.
16번째 함수의 리턴 타입은 void 입니다.
17번째 함수의 리턴 타입은 void 입니다.
18번째 함수의 리턴 타입은 void 입니다.
19번째 함수의 리턴 타입은 int 입니다.
20번째 함수의 리턴 타입은 void 입니다.
21번째 함수의 리턴 타입은 int 입니다.
22번째 함수의 리턴 타입은 void 입니다.
23번째 함수의 리턴 타입은 void 입니다.
24번째 함수의 리턴 타입은 void 입니다.
25번째 함수의 리턴 타입은 void 입니다.
26번째 함수의 리턴 타입은 void 입니다.
27번째 함수의 리턴 타입은 int 입니다.
json_get error : the first argument of json_get should be an array or an object (type : null)
json_get_last_index : the type of json_value is not a JSON_ARRAY nor a JSON_OBJECT28번째 함수의 리턴 타입은 int 입니다.
29번째 함수의 리턴 타입은 void 입니다.
30번째 함수의 리턴 타입은 int 입니다.
31번째 함수의 리턴 타입은 int 입니다.
32번째 함수의 리턴 타입은 int 입니다.
33번째 함수의 리턴 타입은 int 입니다.
34번째 함수의 리턴 타입은 int 입니다.
35번째 함수의 리턴 타입은 int 입니다.
36번째 함수의 리턴 타입은 int 입니다.
37번째 함수의 리턴 타입은 int 입니다.
38번째 함수의 리턴 타입은 int 입니다.
39번째 함수의 리턴 타입은 void 입니다.
40번째 함수의 리턴 타입은 void 입니다.
41번째 함수의 리턴 타입은 void 입니다.
42번째 함수의 리턴 타입은 int 입니다.
* 터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.
```

→ 예외적인 상황이 발생하여 참조 오류가 발생하고 있습니다.

[마무리]

많이 아쉽습니다. 지금까지 했던 과제 중에서 가장 많은 시간을 할애하지 않았나 싶은데... 원하는대로 결과가 따라주지 않았습니다.

누군가에게는 굉장히 간단한 과제일 수도 있는데, 제게는 정말 쉽지 않았습니다. 며칠간 진행하다보니 어디까지 어떻게 생각했었는지 까먹는 경우도 있어서 아래와 같이 진행 상황과 문제점, 아이디어를 작성해가면서 프로그래밍을 진행했습니다.

진행 상황

- 4월 1일까지

- 일단 ast.json 파일을 역직렬화 처리를 해서 txt 파일로 바꿔 저장을 해놓았다.
- 파일 포인터를 이용해서 txt 파일을 읽기 전용으로 열었다.
- fseek으로 파일 포인터를 txt파일 끝으로 이동시키고 ftell로 파일의 총 문자 수를 알아냈다.
- 파일의 모든 문자를 fgets로 문자열로 넣어야하기 때문에, 알아낸 문자 수로 버퍼를 설정해주었다.
- 다시 fseek으로 파일 포인터를 txt 파일 맨 앞으로 이동했다.
- 일단 txt 파일의 문자수가 239893개가 나왔고, fgets는 널문자까지 읽어내므로 버퍼 크기를...
- ⇒ 암튼 파일 불러오기까지 진행

문제점

- 4월 3일

- 문제를 풀어내는 논리는 알겠는데, json 파일을 만드는 과정에서 오류가 있는 듯하다.
- 계속 json_value json_create를 사용하면 계속 출력이 된다..

- 4월 5일

- 근데 생각을 해보니까, 직접 함수를 정의해서 쓰는 경우도 있겠지만은, 외부 라이브러리를 사용한 함수도 있을 것 아니야? 그래서.. 그냥 선언한 것들도 세야할 거 같다.
- 그리고 그냥 선언한 것들 중에서 함수 선언 타입의 이름과, 함수 정의된 것들의 이름을 비교해서 같으면 function_count 에서 1을 빼주면 될 것 같다.

알아낸 것

- 4월 1일

- 함수 개수 추출하려면은 `ext` → `nodetype` == "FuncDef"의 개수를 찾으면 된다 ⇒ 와
- 이게 헛갈렸던 이유가 함수가 깊어지면 깊어질 수록 같은 `nodetype` 등등의 변수? 가 나와서 헛갈렸는데, 각 라인별로 나오는 게 제한이 되어있다.
- 예를 들어서 `ext`안에 있는 `nodetype`은 무조건 `FuncDef` 아니면 함수 선언 등등 이런 거 밖에 안나옴
- 뭔가 실마리가 잡힌다

- 4월 2일

- 함수 개수를 카운팅하려면?

- 4월 5일

- 리턴 타입은 `_nodetype : FuncDef` 인 것들 중에, `decl` 안에, `type`안에 `_nodetyped`이 `FuncDecl`인 것 안에, `type` 안에, `type`의 `name`을 확인하면 된다.
- `decl` → `type` → `type` → `name`

- 4월 8일

- 아니.. 참조에 대한 오류가 파일 열고, 가져와서 배열에 넣은 게 문제가 아니라 그냥 함수 자체를 잘못써서 였다
- `json_value` 변수명 = `json_get`(어쩌고, 어쩌고) 인데, 나는 `json` 파일을 배열로 저장했으나
- `json_get`은 그냥 통째로 가져오는 느낌이라고 한다
- 그래서 계속 참조 오류가 났다고 하는데.. $\pi\pi$ 모르겠다 어렵다

꾸준히 해왔는데, 어느 순간 기록하는 것도 짜증나서 그만뒀버린 것 같네요.. 저는 개발이랑은 맞지 않을 수도 있겠다는 생각이 들었습니다만.. 포기하지 않겠습니다. 제대로 프로그래밍 해본 것이 PCAP 프로그래밍 이후 처음이어서 더 그런 것 같아요.

앞으로 재교육 기간도 있고, 어찌저찌 멘토님께 물어볼 시간은 아직 많이 남았다는 생각이 듭니다 ㅎㅎ.. 이번 과제하면서 정말 무의 상태에서 한 획 정도는 그어본 것 같습니다. 이젠 과제도 아니니 적극적으로 여쭙보겠습니다!

고맙습니다!